

Backend readiness review

Source analysis of routes, GuestContext, localStorage stores, and mock fixtures. The product domain is largely designed. There is no production server, no shared live state across devices, and payments are simulated.

Review date: September 16, 2026 · Exported for sharing: September 23, 2026 · Scores are engineering judgment from code, not runtime metrics.

Verdict	How it works	What 100% means	Gaps	Security	Target API	Roadmap
---------	--------------	-----------------	------	----------	------------	---------

There is no backend to finalize

The app is a Vite + React 19 frontend. GuestContext is the server. localStorage is the database. One browser tab is the restaurant. To go live, extract that domain into APIs and realtime. Connecting remaining screens does not get there.

Update since this review (September 23, 2026)

Local work added a Node HTTP API under /api/v1 for staff login, device registry, and KDS actions, stored in a JSON file (.data/restaurant-api.json). Staff PINs are still compared in plaintext. There is still no Postgres, WebSockets, payment processor, or Hacienda worker. Guest orders, payments, and the floor still live in the browser. The scores and gaps below still describe the product.

~40%

Production-complete

0

Production HTTP APIs / WebSockets

10

localStorage keys (fake DB)

1

Live floor table (#7)

How the app works today

Four portals share one React tree via GuestProvider in App.tsx: guest PWA, server floor, kitchen/bar KDS + cashier, and admin. Mutations (join table, send to kitchen, pay, void, approve guardian) update in-memory SharedState. Table sessions are written to `cr-pat-table-sessions`. Menu, staff, restaurant overlays, and shifts each have their own key. KDS tickets and the guardian queue are not persisted.

Product completeness by layer

Weighted mix used in the overview bar. Domain is ahead of infrastructure. The empty backend segment is intentional: that layer scores 0.



Domain already designed

KEEP

- Table lifecycle: available → seated → ordering → paying → closed.
- Cart → send-to-kitchen → kitchen/bar KDS split by category.
- Guardian (alcohol, price, quantity) with approve/reject and rollback.
- Split: full / equal / pay-my-items with FIFO settlement maps.
- Receipt cycles, Costa Rica 10% service + 13% IVA (independent).
- Staff PIN clock-in, closed-check ledger, shift snapshots.

Fake infrastructure

REPLACE

- GuestContext (~1,200 lines) is a god-object “backend.”
- Checkout waits 1.8s then records a payment locally.
- Admin PIN and staff PIN stored in plaintext localStorage.
- Three restaurant fixtures exist, but catalogs are not scoped by restaurantId.
- Hacienda / DGT / BAC gateway fields are UI-only and not saved.
- PortalSwitcher lets anyone jump guest ↔ kitchen ↔ admin.

localStorage as the current database

KEY	OWNS	SHARED ACROSS TABS?	BACKEND TABLE
cr-pat-table-sessions	Live table cart, guests, payments	Same origin, no live sync	table_sessions, orders, payments
cr-pat-guest-session	QR menu-access token + tableSessionId	Per browser	guest_devices / session tokens
cr-pat-guest-id / name	Anonymous diner identity	Per browser	guests
cr-pat-menu-catalog	Menu CRUD overlay of menuData	Same origin	menu_items (per restaurant)
cr-pat-staff-directory	Staff + 4-digit PINs	Same origin	staff_accounts (hashed PIN)
cr-pat-staff-shift	Open clock-in (one at a time)	Same origin	shifts
cr-pat-shift-ledger / snapshots	Closed checks + Z-style stats	Same origin	checks, shift_closes
cr-pat-restaurant-overlays	Name, tax, brand, SINPE, flags	Same origin	restaurants, settings
cr-pat-table-catalog	16 demo tables / sections	Same origin	tables, sections
cr-pat-admin-session	Manager login flag	Same origin	auth sessions / JWT

Layer scores (frontend logic vs production)

Domain logic (orders, split, tax, guardian)



78%

Guest / staff / KDS / admin UI



82%

Local persistence (localStorage)



45%

Cross-portal wiring in one browser



70%

Real backend / multi-device

0%

Live payments + fiscal invoices

8%

Scores are engineering judgment from code, not runtime metrics.

Why production-complete is ~40%

That number is not “40% of the screens are done.” It answers: could a restaurant operate this product with real guests, real kitchen screens, and real colones tonight? Domain and UI are high, so the demo is strong. Backend and payments are near zero, and those two layers are what “go live” actually means. Weighting them more heavily lands around 38%, rounded to ~40%.

LAYER	NOW	WEIGHT IN 40%	CONTRIBUTION
Domain logic	78%	15%	11.7
Guest / staff / KDS / admin UI	82%	15%	12.3
Local persistence	45%	10%	4.5
Cross-portal wiring	70%	10%	7.0
Real backend / multi-device	0%	25%	0.0
Live payments + fiscal invoices	8%	25%	2.0
Weighted total		100%	~38% → 40%

A simple average would be ~47%

$(78 + 82 + 45 + 70 + 0 + 8) / 6 \approx 47\%$. The 40% figure discounts that because a polished menu screen cannot take a card or drive a second kitchen iPad. Missing backend and payments are blockers, not equal line items.

What 100% means per layer

LAYER	NOW	100% LOOKS LIKE
Domain logic	78%	Payment stores tax + tip + tender; SINPE and empresa invoice are first-class; paid-cycle refunds; checkout lock actually blocks other guests; sequential comprobante numbers.
UI	82%	All catalog tables on the floor; cashier not hardcoded to #7; feature flags drive checkout; Hacienda form saves; no PortalSwitcher in prod; KDS/caja login; PWA install.
Persistence	45%	Every mutation persists (void, cancel, guardian, KDS). KDS + guardian survive refresh. Keys scoped by restaurantId. Two tabs stay in sync via storage events.
Cross-portal wiring	70%	One table catalog feeds floor, QR, KDS, caja, admin. Menu/staff/sessions scoped per restaurant. Live reports only – no WEEKLY_SALES filler.
Backend / multi-device	0%	Postgres + REST + WebSockets. Guest phone, floor tablet, kitchen, bar, caja, admin all see the same session. Signed QR, hashed PINs, server prices.
Payments + fiscal	8%	PSP wallets/cards (no PAN in app), verified SINPE, Hacienda DGT XML v4.4 FE/TE, WhatsApp/email receipts from the existing flags.

100% on every bar means a restaurant can run service, take money, and issue legal invoices. 100% on domain and UI alone is still a prototype.

What is connected vs what is not

Guest, server, KDS, cashier, and admin are wired together only while they share one GuestProvider instance. Open kitchen on a second phone and it will not see the order. That is the core production gap.

Persistence is incomplete even for the prototype

`persistSessions()` runs on send-to-kitchen, staff send, reset, and `updateSession` (join, cart, pay, present bill). It does not run on guest cancel, staff void, cart remove, guardian approve/reject, or KDS advance/bump. Refresh can resurrect voided items. KDS and guardian state always reset on reload.

AREA	STATUS	WHAT TO CHANGE
Floor vs catalog	SPLIT	FLOOR_TABLES is only mesa 7. Admin QR seeds 16 MOCK_TABLES. restaurant.tables is unused. Drive floor from table catalog.
Multi-restaurant	COSMETIC	la-terrazza / soda-marisol / el-patio-azul exist, but menu, staff, and sessions are global. Scope every row by restaurantId.
Payments	SIMULATED	CheckoutModal setTimeout → recordPayment. No PSP, no SINPE capture, no Apple/Google Pay session.
Tax vs payment amount	MISMATCH	Guest sees IVA + servicio on the total; PaymentRecord stores subtotal + tip only. Persist tax lines on the payment.
SINPE + empresa invoice	TYPES ONLY	PaymentMethod includes sinpe; checkout tenders are apple/google/card. Cédula jurídica is collected and discarded.
Hacienda DGT XML 4.4	DEAD UI	SystemSettings gateway + DGT fields never save via saveRestaurantSettings.
Feature flags	ADMIN-ONLY	biometricPay, emailReceipts, whatsappReceipts are toggled but guest checkout ignores them (only dualCurrency is read).
Confirmation receipt	HARDCODED	ORDER_NUMBER is LT-YYYYMMDD-007. Needs sequential comprobante per restaurant.
Auth	CLIENT PIN	KDS and cashier have no login. PortalSwitcher bypasses role gates. Hash PINs server-side; issue JWTs.
Cashier POS	HARDCODED #7	getTableSession("7") for today's payments. Must iterate live sessions / checks.
Admin 7-day report	HYBRID MOCK	buildAdminReport fills historic days from WEEKLY_SALES, live only for "Hoy".
PWA / QR	PROTOTYPE	HashRouter, no service worker, QR via api.qrserver.com. Need HTTPS origin, signed table tokens, installable manifest.

Security blockers before any real restaurant

Must not ship

- Plaintext 4-digit PINs in localStorage (demo PIN 0000 for manager).
- Fake BAC API key field labeled sk_live_bac_... in settings.
- CardDetails type (PAN/CVV) – never collect cards in the PWA.
- Unauthenticated /kds/* and PortalSwitcher in production builds.

Must add on the server

- Signed QR: restaurantId + tableId + rotating nonce, not guessable /m/7.
- Server-authoritative prices (never trust client unitPrice).
- Optimistic concurrency on table session version for split pay.
- PCI via PSP hosted fields / wallets only; SINPE confirmation codes server-side.

Recommended backend shape

Keep the existing TypeScript domain (billing.ts, guardian.ts, kds.ts, tax.ts, order.ts) as shared packages. Replace GuestContext mutations with a thin API client + WebSocket. Do not rewrite the UI first.

Suggested stack

TypeScript API (Fastify or NestJS) + PostgreSQL + Redis pub/sub + WebSockets. Object storage for menu photos. PSP for Apple/Google/card (BAC / CyberSource or equivalent CR acquirer). Separate worker for Hacienda DGT XML v4.4. One tenant = one restaurant row; never share catalogs across restaurants.

Core entities

ENTITY	MAPS FROM	REALTIME?
Restaurant, Settings, Brand	restaurants.ts + restaurantConfig	No
Table, Section	tableCatalog (not FLOOR_TABLES)	Status yes
MenuItem, ModifierGroup	menuCatalog	Availability yes
StaffAccount, Shift	staffDirectory + staffSession	Shift yes
TableSession, Guest	GuestContext TableSession	Yes
Order / CartItem / KdsTicket	sendToKitchen + kds.ts	Yes
GuardianReview	guardianQueue	Yes
Payment, TaxLine, Tip	recordPayment + tax.ts	Yes
ElectronicInvoice (FE/TE)	CheckoutModal invoiceType (unused)	Async
ClosedCheck, ShiftClose	shift ledger / snapshots	No

Minimum API surface (replace GuestContext)

METHOD	PURPOSE
POST /guest/sessions/join	QR token → guestId, table session, menu access
POST /tables/:id/cart	Add/update/remove lines; server prices
POST /tables/:id/send	Create order, guardian or KDS tickets
POST /orders/:id/cancel-item	Undo window + kitchen-not-started
POST /guardian/:id/approve reject	Staff; reject rolls back sent lines
POST /kds/:ticketId/advance	pending → preparing → ready → delivered
POST /tables/:id/split-lock	Authoritative remaining + lock TTL
POST /payments/intents	Create PSP/SINPE intent; webhook completes recordPayment
POST /staff/clock-in out	PIN verify server-side; one open shift policy
POST /tables/:id/present-bill clear	staffPresentBill / staffClearTable rules
CRUD /admin/menu /staff /tables /settings	Replace local overlays
GET /admin/reports	Replace adminReports + kill WEEKLY_SALES
WS /ops	table.updated, kds.ticket, guardian.queued, payment.captured

Frontend extraction (do this before writing routes)

Introduce a TableOps interface with the same methods GuestContext already exposes (joinTable, sendToKitchen, recordPayment, ...). Today implement it with the current React state. Tomorrow swap in HTTP+WS without rewriting MenuScreen or FloorPlan.

Path to 100% complete

Do not treat “add a backend” as one task. Ship in this order so the restaurant can run table 7 for real, then scale to the 16-table floor.

- 1 **P0** Extract TableOps from GuestContext; persist every mutation; storage event or BroadcastChannel so two tabs stay in sync (proves the contract).
- 2 **P0** One table source of truth: catalog drives floor + QR + cashier. Remove FLOOR_TABLES vs MOCK_TABLES split. Scope menu/staff/sessions by restaurantId.
- 3 **P0** Real auth: hashed staff PINs, manager JWT, no PortalSwitcher in prod, KDS/cashier require a station session.
- 4 **P0** Signed QR tokens + BrowserRouter on HTTPS. Guest cannot guess another table. Bind menu access to sessionId server-side.
- 5 **P1** Postgres + REST for sessions/orders/KDS/guardian. WebSocket fan-out to guest phones, floor tablet, kitchen, bar, caja.
- 6 **P1** Payment intents: wallets + card via PSP (never PAN in app). SINPE Móvil as a first-class tender with reference verification. Store tax lines + tip + comprobante number.
- 7 **P1** Hacienda DGT XML v4.4 worker (FE/TE, cédula jurídica). Persist settings for real; send WhatsApp/email receipts from flags.
- 8 **P2** Full floor (all catalog tables), Z-report from ledger not table 7, live-only analytics, 86'ing items, image CDN.
- 9 **P2** Real PWA: web app manifest, service worker for menu shell, offline "reconnect" banner, printer/cash drawer at caja.
- 10 **P3** True multi-tenant (many restaurants), station pairing codes, audit log, refunds/voids on paid cycles, load tests for concurrent split pay.

What not to do

Keep GuestContext as the contract

- Do not bolt a generic CRUD backend onto the current screens without extracting TableOps. GuestContext will stay the source of truth and the API will drift.

Stay out of PCI scope

- Do not collect card numbers in CheckoutModal to “finish payments.” That would make the app PCI-scope overnight.
- Do not keep HashRouter + public /m/:tableId once real money moves. Table IDs are currently guessable.

Source: local codebase review · September 16, 2026 · frontend-only prototype, with a September 23 note on the staff/device/KDS JSON API that started afterward. Open this file in any browser. No server required.